

ML System Design Interview Prep:

Regulatory AI Question-Prediction System

Complete 45-Minute Whiteboard Guide + Concept Encyclopedia

System: monitors regulatory changes worldwide, converts them into a queryable knowledge substrate (vector index + knowledge graph + obligation store), and powers a Predictive Analyst that predicts which questions a human inspector/analyst is likely to ask — with cited draft answers and evidence packs.

Built on AWS + OpenAI.

PART 1 — The 45-Minute Whiteboard Talk Track

Minute 0:00–0:05 — Clarify the problem (don't skip this)

Ask before you draw. Strong candidates ask 3–5 sharp questions:

1. "Who are the users — QA analysts preparing for inspections, or inspectors themselves?" (Answer: analysts/compliance officers; inspectors are the *subjects* being modeled.)
2. "What does 'predict questions' mean concretely — a ranked list per site?" (Yes: top-k ranked questions with rationales.)
3. "Which regulatory domains? What freshness do we need — minutes, hours?" (Pharma/life-sciences assumed; publication-to-alert within hours.)

4. "Scale: how many sources, documents, analysts?" (Hundreds of sources, thousands of docs/day, tens of analysts — but design for 10x.)

Then state your assumption out loud: *"I'll design for pharma — FDA-style surprise inspections, 483s, GMP — and note where it generalizes."* Interviewers love explicit scoping.

Minute 0:05–0:10 — Requirements

Functional: - Ingest regulatory sources (FDA, EMA, ICH, MHRA, PMDA, Federal Register...) continuously - Detect new/changed documents; classify material vs editorial changes - Extract structured obligations (who must do what, by when) from regulatory text - Build a queryable substrate: hybrid vector index + knowledge graph + obligation store - Ingest internal corpus (SOPs, 483s, CAPAs, training records) with access control - Predict ranked questions per (site, trigger, inspector?) context, with human-readable rationales - Draft answers with citations + evidence packs; abstain when evidence is missing - Feedback loop: log what was actually asked → retrain

Non-functional: - Freshness: publication → alert under 4 hours - Latency: live re-rank p99 < 2s; ranked questions never wait for answer generation (stream answers) - Auditability: every prediction/answer traceable to source docs, model versions (Part-11-style) - Security: ACL enforcement at retrieval; PII redaction; zero data retention on LLM APIs - Recall-first change detection: missing a guidance update is a compliance incident; false positives are cheap triage - Availability 99.9% for the analyst UI; degrade gracefully (cached predictions if ranker is down)

Minute 0:10–0:20 — High-level design (draw this)

Draw four boxes left to right, with a feedback loop underneath:

Sources —▶ [1. RegMonitor] —▶ [2. Knowledge Prep] —▶ [3. Predictive Analyst] —▶ Analyst UI

ingest/crawl parse/extract/KG/index
context→candidates→rank→RAG



label → retrain] — [4. Feedback loop: log →

Narrate each box in one breath, naming real services: 1. **RegMonitor:**

EventBridge-scheduled Lambda connectors → SQS → S3 raw lake (versioned, Object Lock) → Step Functions diff pipeline (hash → SimHash → OpenAI judge for semantic materiality) → DynamoDB change registry → SNS alerts, impact-scored.

2. **Knowledge Prep:** S3 events → Textract → section-aware chunking (Fargate) → OpenAI structured-output obligation extraction with span-grounding validation → Neptune knowledge graph + OpenSearch Serverless hybrid index (text-embedding-3-large + BM25). Internal docs ingested with ACL tags; Batch API for backfills. 3. **Predictive Analyst:** SageMaker Feature Store context → three candidate paths (ANN retrieval over question bank, LLM generation with obligation-ID grounding validation, deterministic templates) → LightGBM LambdaMART ranker → MMR diversification → RAG answers with citation enforcement + NLI faithfulness gate + abstention → evidence packs. Served via API Gateway → Fargate, Redis cache, nightly + event-driven precompute. 4. **Feedback loop:** Kinesis Firehose → S3 Parquet lake → SageMaker Pipelines retrain → Model Registry → shadow/canary/prod. North-star metric: hit rate@10.

Say this sentence: *"The architecture separates concerns cleanly: monitoring is a recall-optimized detection problem, knowledge prep is a precision-optimized extraction problem, and prediction is a learning-to-rank problem — each gets its own eval."*

Minute 0:20–0:33 — Deep dives (let the interviewer pick; default to these two)

Deep dive A — Question prediction as learning-to-rank (the ML heart). -

Formulation: context $C \rightarrow$ ranked questions; listwise/pairwise loss; two modes (precompute vs live). - Labels: inspection logs (gold, scarce) + analyst chat logs + **483 reverse-engineering** ("what question surfaced this observation?" — 10x's your data) + expert-reviewed synthetic. Hard negatives from similar-but-different contexts. Temporal splits, grouped-by-site ablation. - Features (name 5–6): embedding similarity, recency-decayed delta impact $\Sigma \text{ impact} \cdot e^{-(\text{days}/60)}$, facility topic risk (483s/inspections), inspector affinity from public 483s, KG applicability bit, novelty vs already-asked. - Position-bias correction with IPS; LightGBM LambdaMART; isotonic calibration; MMR $\lambda \approx 0.7$; epsilon-greedy exploration slots. - Rationale per question from top feature contributions — the "human predictable" requirement.

Deep dive B — Grounding: extraction + RAG without hallucinations. -

Obligation extraction: two-pass (deontic sentence classification $\rightarrow$ structured extraction), schema {subject, action, conditions[], deadlines[], source_span}; **span-grounding validator** rejects anything not fuzzy-matched in source text; LLM critic + human review for high-impact items. - RAG: hybrid retrieval (ACL pre-filtered) $\rightarrow$ rerank $\rightarrow$ generate with citation-enforced structured output $\rightarrow$ **NLI entailment gate** per sentence (≥ 0.7) $\rightarrow$ repair once $\rightarrow$ drop $\rightarrow$ abstain if < 2 sentences survive. - Say: *"In a regulated industry, abstention is a feature — a missing answer with an evidence gap flagged is more valuable than a fluent hallucination."*

Minute 0:33–0:40 — Scale, bottlenecks, trade-offs

Numbers to have ready (say "rough, then 10x headroom"): - ~200 sources; 500–2,000 docs/day raw, ~20–50 material - Internal corpus 1–10M docs $\rightarrow$ 20–50M chunks; 1536-d embeddings $\approx$ 6KB/chunk $\rightarrow$ ~30–60GB quantized in OpenSearch -

Judge/classifier calls: low thousands/day — negligible cost - Serving: tens of QPS; p99 < 2s re-rank on cache hit; generation is async/streamed - Ranker training: 100k–1M pairs, minutes on LightGBM

Bottlenecks & fixes: LLM generation latency (the long pole) → precompute + cache + stream; Textract on huge PDFs → async Batch; OpenSearch reindex on embedding upgrade → blue/green versioned indexes.

Trade-offs to state proactively: - Recall over precision in change detection (compliance risk asymmetry) - Knowledge graph AND vector DB (relations need the graph; semantics need vectors — neither alone suffices) - LambdaMART over a neural ranker (fast to train/iterate, interpretable features, plenty for this signal regime) - Precompute + cache over pure real-time (analyst QPS is low; freshness matters more than millisecond inference)

Minute 0:40–0:45 — Evaluation, rollout, close

- Offline: P@5/10, Recall@20, nDCG@10, MRR on temporal splits; extraction F1; citation precision ≥ 0.95
 - Online: hit rate@10 (north star), adoption, time-to-evidence; guardrails (abstention 5–15%, latency)
 - Ranker iteration via off-policy SNIPS on exploration logs before any rollout
 - Rollout: Phase 0 ingestion+alerts → Phase 1 RAG copilot → Phase 2 KG+retrieval prediction → Phase 3 generative ranker + inspection mode → Phase 4 hardening
 - Close with intellectual honesty: *"Hardest parts: label scarcity for the ranker, keeping the KG current as regulations version, and preventing feedback-loop popularity bias. That's where I'd put the best people."*
-

What to physically draw

1. The four-box diagram with the feedback loop (top third of board — keep it up the whole time, point back to it).
 2. Inside box 3, the prediction pipeline: context → [retrieval | generation | templates] → ranker → MMR → answers+evidence.
 3. A small feature table for the ranker (5–6 features with one-line definitions).
 4. The RAG grounding chain: retrieve → rerank → generate+citations → NLI gate → abstain?.
 5. Latency budget bar for live mode: features 10ms | cache 20ms | ANN 50ms | rank 30ms | answers stream async.
-

Interviewer Q&A bank — 18 likely follow-ups with strong answers

1. "Why not just RAG over the regulations? Why predict questions at all?"

RAG answers questions the analyst already thought to ask. The value here is *anticipation*: in a surprise inspection there's no time to think of everything.

Predicting the question set shifts work left — evidence packs are pre-built, and gaps ("high likelihood, no evidence") surface before the inspector finds them. RAG is a component (answering), not the product.

2. "How do you evaluate question prediction offline?" Temporal split — train on inspections up to 2024, test on 2025+. Metrics: Precision@5/10, Recall@20, nDCG@10, MRR against the questions actually asked. Slice by new vs known inspector, routine vs for-cause, and unseen sites (grouped split). Plus human eval: QA veterans rate top-10 lists for relevance/actionability, tracking inter-rater kappa.

3. "Cold start — new regulation with zero history, new site, unknown

inspector?" Three backoffs: new regulation → template path dominates (every obligation deterministically yields questions) + population priors by topic; new site → company prior, then archetype prior (sterile fill-finish sites resemble each other); unknown inspector → population topic distribution. The system degrades to sensible priors, never to nothing.

4. "Why a knowledge graph AND a vector database?" They answer different queries. Vectors: "find me text semantically like this." Graph: "which of *my* processes does this obligation apply to, and which facilities run those processes?" Multi-hop applicability reasoning is unreliable in pure vector search. The graph also versions regulations (SUPERSEDES edges) so predictions expire cleanly.

5. "How do you prevent hallucinations in obligation extraction?" Four gates: (a) structured output schema constrains shape; (b) span-grounding validator — every extracted field must fuzzy-match a source span or the obligation is discarded; (c) LLM critic second pass; (d) human review queue for anything patient-safety-related. Measured by extraction F1 on a golden set, not vibes.

6. "Why LambdaMART instead of a neural ranker?" Signal regime and iteration speed. With ~11 dense features and 100k–1M pairs, gradient-boosted trees train in minutes, are robust to unnormalized features, and give interpretable feature contributions for the rationales. A neural ranker earns its complexity only when you have raw-text cross-encoding at scale — we use cross-encoders at the rerank stage instead, where they matter.

7. "How do you handle position bias in click/ask logs?" Inverse propensity scoring: weight each training example by $1/(\text{examination probability at its shown position})$, estimated from historical examination rates by position. Without this the ranker just learns "top of the old list = good."

8. "What if the analyst over-relies on the predictions?" Product mitigation, not just ML: every prediction shows its rationale and linked evidence; the UI shows coverage stats ("12 obligations with no predicted question — review manually");

training emphasizes assistive framing. The abstention path ("evidence gap flagged") is deliberately designed to trigger human investigation.

9. "How do you keep the system fresh when regulations change intraday?"

Event-driven invalidation: a material change event recomputes impact, updates the KG, invalidates Redis entries for affected sites, and triggers precompute for those sites. Publication → refreshed predictions in under 4 hours, typically under 1.

10. "Data privacy — you're sending pharma docs to OpenAI?" Tiered: public regulatory content flows freely. Internal GxP docs only under an enterprise zero-data-retention agreement, PII redacted pre-call, all calls logged, chunk-level ACLs enforced at retrieval so the model never sees what the user can't. If a customer requires it, the architecture supports swapping the LLM provider behind the orchestrator interface.

11. "How do you detect the ranker going stale?" Three signals: offline — scheduled backtests on recent months (nDCG decay alarm); online — hit rate@10 trend, exploration-slot win rate rising (means the exploitative ranking is missing things); feature drift — PSI/KS tests on top features. Any alarm triggers retraining; severe decay triggers automatic rollback to the previous model version.

12. "Walk me through a single live request, with latencies." API Gateway → Lambda auth (~20ms) → context hash → Redis: hit returns precomputed top-20 in ~50ms total. Miss: Feature Store online read (~10ms) + ANN retrieval (~50ms) + ranker (~30ms) + MMR → questions in well under 500ms; generation path (2–4s) runs async for novel candidates; answers stream over WebSocket as ready. p99 target < 2s for questions, answers streaming.

13. "What breaks at 10x scale?" Textract throughput (→ async Batch + reserved capacity), OpenSearch indexing lag (→ decouple via queue, scale OCU), LLM rate limits on generation bursts (→ request hedging + caching + Batch API for offline),

KG write contention (→ Neptune reader/writer split, batched writes). None are architectural — all are capacity.

14. "How do you version everything for audit?" Immutable S3 Object Lock log of every prediction and answer: input hash, model versions (ranker, embeddings, LLM), evidence doc IDs, timestamp, user. Model Registry versions rankers; embedding/index versions are blue/green. A prediction from March must be reproducible in September.

15. "Why not fine-tune an embedding model instead of using OpenAI's?"

Pragmatic sequencing: off-the-shelf `text-embedding-3-large` is strong enough for v1, and fine-tuning adds a retraining/versioning burden across every index. If domain eval shows a gap (e.g., regulatory jargon), fine-tune a bi-encoder with InfoNCE on (context, question) pairs — but measure first. Don't pay complexity tax upfront.

16. "How do you avoid feedback-loop popularity bias?" Epsilon-greedy exploration slots (2 of 20), novelty features in the ranker, temporal (not random) evaluation splits, and off-policy SNIPS estimation on exploration logs before promoting any ranker. Popularity is a feature, not the objective.

17. "What would you build first with 6 weeks and 3 engineers?" Phase 0: ingestion → lake → hash/SimHash diffs → analyst alert inbox. It delivers standalone value (never miss a guidance) and every later system consumes its outputs. No ML needed on day one — earn the right to do ML by nailing the data foundation.

18. "What's the hardest part of this system, honestly?" Label scarcity for the ranker (solved partially by 483 reverse-engineering, never fully), KG maintenance as regulations version and cross-reference each other, and the human-factors problem: predictions must be *trusted but not over-trusted*. The technology is tractable; the data flywheel and the UX are where it lives or dies.

PART 2 — Concept Encyclopedia

Every concept used in this design, explained at interview depth: what it is, why it's here, and the trade-off or detail an interviewer will probe. Organized by theme.

A. Design framework

Functional vs non-functional requirements. Functional = what the system does (ingest, detect, predict, answer). Non-functional = how well (latency, freshness, availability, auditability, security). *Here:* the NFRs drive the architecture — recall-first detection, $p99 < 2s$ re-rank, immutable audit log, ACLs. Interviewers check that your NFRs actually shape decisions, not decorate the doc.

Capacity estimation. Back-of-envelope math: $\text{sources} \times \text{docs/day} \times \text{size} \rightarrow \text{storage}$; $\text{QPS} \times \text{latency} \rightarrow \text{serving capacity}$; $\text{tokens/day} \rightarrow \text{LLM cost}$. *Here:* $200 \text{ sources} \times \sim 1,000 \text{ docs/day} \times 50\text{KB} \approx 10\text{GB/day raw}$; $20\text{--}50\text{M chunks} \times 6\text{KB} \approx 30\text{--}60\text{GB quantized vectors}$. Always state assumptions, then add 10x headroom. The point isn't precision — it's showing you size before you build.

API design. REST for CRUD-like operations (alert subscriptions), WebSocket/SSE for streaming answers. Version your APIs; idempotency keys on anything retried. *Here:* prediction API takes a context object, returns ranked questions; answer generation streams over WebSocket so questions never wait for answers.

B. Ingestion & streaming

Connectors (source adapters). One adapter per source family (RSS, REST API, HTML crawl) behind a common interface emitting normalized documents. *Here:*

FDA, EMA, ICH each get a connector; new sources plug in without touching the pipeline. Isolates source quirks (auth, pagination, JS rendering) from downstream logic.

Polling vs webhooks. Polling (we ask on a schedule) is universal but latent; webhooks (they push to us) are instant but require source support. *Here:* polling via EventBridge Scheduler, since regulators don't offer webhooks. Cadence per source velocity — hourly for fast dockets, daily for slow ones.

ETag / If-Modified-Since. HTTP conditional-request headers: the server returns 304 Not Modified if content is unchanged, saving bandwidth and polite-crawl budget. *Here:* first-line change detection, before any bytes are stored.

Rate limiting & robots.txt. Respect crawl-delay, per-domain concurrency caps, and exponential backoff on 429s. *Here:* getting banned by a regulator's site is an embarrassing, avoidable outage. Token-bucket per domain in the connector.

Message queues (SQS). Durable buffers decoupling producers from consumers; absorb bursts, allow independent scaling. *Here:* between connectors and parsers, and before the embedding stage. Visibility timeouts must exceed max processing time or work gets double-processed.

Dead-letter queues (DLQ). After N failed processing attempts, poison messages move to a DLQ instead of blocking the queue. *Here:* malformed PDFs, parser crashes. Alarmed, inspected, replayable — never silently dropped.

Idempotency. Processing the same message twice must not corrupt state — key writes by content hash (doc_id + version), not by auto-increment. *Here:* crawler retries and SQS at-least-once delivery make duplicates certain; dedup keys make them harmless.

At-least-once vs exactly-once. Distributed queues guarantee at-least-once delivery; exactly-once requires idempotent consumers or transactions. *Here:* we accept at-least-once + idempotent writes everywhere — simpler and sufficient.

Backpressure. When downstream is slower than upstream, you need a strategy: queue (absorb), shed (drop with policy), or slow the producer. *Here:* SQS depth alarms trigger parser autoscaling; if depth keeps growing, alert — don't silently shed regulatory documents.

Schedulers (EventBridge Scheduler / Airflow). Cron-like triggers for periodic work. *Here:* connector cadences, nightly precompute, weekly retraining. Prefer managed scheduler over cron-on-a-box (no single point of failure, audit trail of invocations).

C. Storage & data

Object storage (S3). Infinitely scalable blob store; the right home for raw documents, Parquet lakes, model artifacts. *Here:* raw lake (immutable), chunk store, audit log, backups. Lifecycle policies move old versions to cheaper tiers.

Partitioning. Organizing keys as `source=/date=/doc_id=` so queries scan only relevant prefixes. *Here:* raw lake partitioned by source and date — a backfill for one source never scans the whole bucket. Partition pruning is the difference between a \$2 and \$200 Athena query.

Parquet + data lake. Columnar format: compressed, splittable, predicate pushdown. *Here:* interaction events land via Firehose as Parquet — cheap to store, fast to scan for retraining and analysis.

Catalog (Glue Data Catalog). A metastore mapping S3 paths to table schemas so Athena/Spark can query them with SQL. *Here:* makes the event lake and change registry queryable without building a warehouse.

DynamoDB data modeling. Single-table design: choose partition/sort keys for your access patterns (e.g., `PK=SITE#id`, `SK=CHANGE#timestamp` for "recent changes for a site"). *Here:* change registry keyed by (doc_id, version); feature online store keyed by site_id. Design for the queries you'll actually run; scans are

a design smell.

Versioning & immutability (Object Lock / WORM). Write-Once-Read-Many: objects can't be altered or deleted until retention expires. *Here:* raw regulatory documents and the prediction audit log are WORM — a regulator or litigator can trust the record.

Hot/warm/cold tiers. Hot: millisecond access (Redis, DynamoDB). Warm: seconds (S3 Standard, OpenSearch). Cold: minutes–hours (Glacier). *Here:* precomputed predictions are hot; the raw lake is warm; compliance archives are cold.

D. Text representation

Tokenization. Splitting text into model-consumable units (subwords for BPE/WordPiece). *Here:* matters for cost math (OpenAI bills per token), context-window budgeting, and chunk sizing. Rough rule: 1 token $\approx$ $\frac{3}{4}$ word in English.

Embeddings. Dense vectors where semantic similarity $\approx$ geometric closeness. *Here:* `text-embedding-3-large` for documents, chunks, contexts, and questions — the shared representation powering retrieval, dedup clustering, and drift detection.

Cosine similarity. Normalized dot product, in $[-1, 1]$; the workhorse similarity for embeddings. *Here:* thresholds everywhere — ≥ 0.90 for question dedup, < 0.92 drift flag for changed sections. Thresholds are tuned on labeled data, not guessed.

Matryoshka embeddings. Training embeddings so prefixes of the vector remain useful — you can truncate 3072→1536 dims with minimal quality loss. *Here:* halves vector storage and speeds ANN search; the cost/latency sweet spot.

Chunking. Splitting documents into retrievable units. Strategies: fixed-window (simple, breaks semantics), section-aware (respects headings — preferred), semantic (split on topic shifts). *Here:* section-aware, 512–1024 tokens with

overlap; tables serialized to markdown; every chunk carries `{doc_id, section_path, page, access_tier}` metadata. Bad chunking is the silent killer of RAG quality.

E. Search

Inverted index & BM25. The classic keyword search: maps terms → documents, scored by term frequency saturated (k1), inverse document frequency, and length normalization (b). *Here:* half of the hybrid retrieval — BM25 catches exact regulation numbers and quoted phrases ("21 CFR 211.68") that embeddings smear.

Approximate nearest neighbor (ANN) / HNSW. Exact vector search is $O(n)$; HNSW builds a navigable small-world graph for sub-linear approximate search with tunable recall (`ef_search`). *Here:* OpenSearch Serverless vector engine; `ef_search` ≈ 200 for the question bank. Alternatives: IVF-PQ (more memory-efficient, slightly lower recall).

Hybrid search & fusion. Combine lexical + vector scores: weighted sum $\alpha \cdot \text{vector} + (1-\alpha) \cdot \text{BM25}$ (α tuned on eval) or Reciprocal Rank Fusion. *Here:* regulation IDs need BM25; paraphrased concepts need vectors. Neither alone is enough — say this in the interview.

Query rewriting. Transforming the user's (or question's) query before retrieval: expansion, hypothetical document embeddings (HyDE). *Here:* used lightly — the question text is already a good query; rewriting helps for short/ambiguous ones.

Rerankers / cross-encoders. A second-stage model scoring (query, document) pairs jointly — far more accurate than bi-encoder cosine, far more expensive, so applied only to top-30. *Here:* before answer generation; also distillable into training data for the bi-encoder later.

F. Large language models

Autoregressive LMs & reasoning models. Next-token predictors; "reasoning" variants allocate extra compute to chain-of-thought before answering, dramatically better at multi-step judgment. *Here:* reasoning models for the materiality judge, obligation extraction, and question generation; smaller/faster models for classification and reranking.

Temperature / top-p. Sampling controls: temperature scales logit randomness (0 = greedy), top-p (nucleus) samples from the smallest set covering probability p. *Here:* 0.7 for diverse question generation (we want variety), ~0 for extraction/classification (we want determinism).

Context window & packing. The max tokens per call; effective use means budgeting: context summary + exemplars + obligations within limits, most important content positioned where attention is strongest. *Here:* ~2k-token prompt budget for generation; long guidance docs are pre-chunked, never stuffed whole.

Structured outputs. Constraining generation to a JSON schema (guaranteed valid). *Here:* used for the judge verdict, obligation records, and generated questions — turns free text into pipeline-compatible data. Validation gates then check *content* (grounding), not just shape.

Function calling. Letting the model invoke typed tools mid-reasoning. *Here:* the analyst copilot can call `get_obligations(site)`, `fetch_evidence(question_id)` — the model orchestrates, tools execute.

Few-shot prompting. In-context examples steering format and quality. *Here:* 8 MMR-diverse exemplar (context → questions) pairs in the generation prompt; retrieved dynamically per context, not hardcoded.

LLM-as-judge. Using a strong model to evaluate outputs (relevance, faithfulness) where human labels are scarce. *Here:* materiality verdicts, extraction critic pass,

offline answer grading — always calibrated against a human-labeled sample, never trusted blindly.

Batch API. Async LLM API at ~50% discount for non-urgent work (24h turnaround). *Here:* backfills — re-extracting obligations across the corpus, synthetic question generation, bulk judging. Real-time path never uses it.

Fine-tuning. Training a model further on task data. *Here:* gpt-4o-mini fine-tuned for multi-label topic classification (cheap, fast, good enough). Deliberately *not* used for embeddings (OpenAI doesn't offer it; and off-the-shelf is sufficient until eval proves otherwise).

Hallucination taxonomy & mitigations. Intrinsic (contradicts source) vs extrinsic (unverifiable). Mitigations stack: retrieval grounding → constrained/structured generation → span validation → NLI entailment gate → abstention. *Here:* all five are implemented; in a regulated domain, abstention is a feature.

Natural language inference (NLI). Classifying sentence pairs as entailment / contradiction / neutral. *Here:* DeBERTa-v3 MNLI checks every generated answer sentence against its cited spans (threshold 0.7) — the faithfulness gate before anything reaches the analyst.

G. Knowledge graphs

Property graphs. Nodes and edges both carrying key-value properties (vs RDF triples). *Here:* Neptune stores Regulation, Obligation, Facility, Inspection, Question nodes with rich properties; edges like APPLIES_TO carry {confidence, effective_date}.

Graph query languages (openCypher / Gremlin). Declarative (Cypher: `MATCH (f:Facility) -[:RUNS]->(p:Process)<-[:APPLIES_TO]-(o:Obligation)`

vs traversal-based (Gremlin). *Here*: multi-hop applicability queries — "which obligations apply to this site via its processes" — that vector search can't express.

Entity resolution. Deciding that "21 CFR Part 211", "21CFR211", and "cGMP for finished pharmaceuticals" are the same regulation. *Here*: rules + embedding similarity + human review for high-impact merges; wrong merges corrupt every downstream query, so precision matters more than recall.

Ontology / schema design. The fixed set of node labels and edge types — your contract with every writer and reader. *Here*: 13 node types, 9 edge types; versioned, because adding an edge type later shouldn't break existing queries.

H. Ranking & recommendation

Learning to rank: pointwise / pairwise / listwise. Pointwise: predict a relevance score per item (regression/classification). Pairwise: learn "A > B" preferences. Listwise: optimize the whole ranked list directly (e.g., LambdaMART's NDCG gradients). *Here*: LambdaMART (listwise) — we care about the ordering of the top-k, not absolute scores.

LambdaMART / LightGBM. Gradient-boosted decision trees where each tree fits the "lambda" gradients of a ranking metric. LightGBM's histogram binning makes it fast on millions of pairs. *Here*: ~500 trees on 11 dense features; trains in minutes on SageMaker; interpretable enough to power rationales.

Ranking features. The signals the ranker combines — similarity scores, popularity priors, recency, risk, applicability. *Here*: the 11 features from the deep dive. Interview tip: always be able to name your top 5 features and why each earns its place.

Position bias & IPS. Users examine top positions more, so clicks overstate top-item quality. Inverse Propensity Scoring weights examples by $1/P(\text{examined} | \text{position})$. *Here*: estimated from historical examination-by-position curves;

without it the ranker learns "old rank = good."

Calibration (Platt / isotonic). Mapping raw scores to true probabilities. Platt = logistic fit; isotonic = non-parametric stepwise fit (more flexible, needs more data). *Here:* isotonic on validation — because "score 0.8" should mean ~80% ask probability when we show confidence to analysts.

MMR (Maximal Marginal Relevance). Greedy diversification: iteratively pick the item maximizing $\lambda \cdot \text{relevance} - (1-\lambda) \cdot \text{max_similarity_to_already_picked}$. *Here:* $\lambda \approx 0.7$ over top-100 → final 20; prevents five paraphrases of the same audit-trail question.

Exploration vs exploitation (epsilon-greedy). Reserve ϵ of slots for uncertain items to gather learning signal, exploit the rest. *Here:* 2 of 20 slots; exploration outcomes feed off-policy evaluation. The antidote to popularity bias and filter bubbles.

Ranking metrics. Precision@k (fraction of top-k relevant), Recall@k (fraction of all relevant captured in top-k), MRR (1/rank of first relevant — good when one answer suffices), nDCG@k (discounted, graded relevance — the gold standard for ranked lists), MAP (mean average precision). *Here:* nDCG@10 primary offline; hit rate@10 (did the inspector actually ask it) primary online.

I. Training data

Weak supervision. Programmatic labeling when hand labels are scarce (heuristics, patterns, distant supervision). *Here:* 483 reverse-engineering — observations imply the questions that surfaced them. Noisy but abundant; always validated on a human-reviewed sample.

Synthetic data. LLM-generated training examples. *Here:* sampled (site × delta × inspector archetype) contexts → generated questions → QA-specialist review; only reviewed items enter the gold set. Unreviewed synthetics get low training weight

— synthetic data is a supplement, not a foundation.

Hard negatives. Negatives the model currently gets wrong — far more informative than randoms. *Here:* real questions asked in similar-but-different contexts (same topic, different site), mined by finding high-scoring false positives from the current model. Retrain → re-mine → repeat.

Contrastive learning / InfoNCE. Pull positive pairs together, push negatives apart in embedding space: $-\log(\exp(\text{sim}+)/\sum \exp(\text{sim}))$. *Here:* the (optional) bi-encoder fine-tuning stage if off-the-shelf embeddings prove insufficient on domain eval.

Label noise & PU learning. Real logs are noisy: asked-but-unlogged questions look like negatives. Positive-Unlabeled learning treats unlabeled data carefully instead of as hard negatives. *Here:* pragmatic version — downweight unlabeled impressions rather than full PU machinery; ranking objectives are naturally robust to some noise.

Temporal splits & leakage. Always split by time (train past → test future), never randomly — random splits let the model memorize the future. Grouped splits (by site) test generalization to unseen entities. *Here:* both; plus point-in-time feature joins so no feature uses information from after the prediction moment. Leakage is the #1 way offline metrics lie.

Dedup via clustering. Embedding + agglomerative clustering (cosine ≥ 0.90) → canonical items; labels aggregate at cluster level. *Here:* applied to questions (50–200k raw → canonical bank) and to near-duplicate regulatory documents (SimHash pre-filter).

J. Feature platform

Feature Store (online vs offline). Online: low-latency KV reads for serving (DynamoDB-backed, ms). Offline: historical, point-in-time-correct datasets for

training (S3/Glue). *Here*: SageMaker Feature Store; the same feature definitions serve both — that's the whole point.

Point-in-time joins. Training rows joined to feature values *as of the event timestamp*, not current values. *Here*: a 2024 training example must see 2024's CAPA counts, not today's. Get this wrong and your model learns from the future.

Training-serving skew. Features computed differently (or drifting) between training and serving. *Here*: mitigated by shared definitions + monitoring feature distributions in both paths; PSI alarms on divergence.

TTLs & invalidation. Time-to-live on cached/precomputed features; event-driven invalidation on state changes. *Here*: delta features recomputed on material-change events; CAPA counts invalidated on QMS updates — not just nightly.

K. Evaluation

Offline vs online. Offline: fast, cheap, on historical data (can lie via leakage/bias). Online: slow, expensive, ground truth (A/B tests, hit rate). *Here*: offline gates every model change; online hit rate@10 is the north star; never ship on offline metrics alone.

Golden sets. Curated, human-verified evaluation datasets, versioned and held out from training. *Here*: temporal inspection-question sets, extraction F1 set, citation-precision set. Guard them like production data — contamination invalidates everything.

Human eval & Cohen's kappa. Experts rate outputs on rubrics (relevance, actionability, faithfulness); kappa measures inter-rater agreement (0.6+ is respectable). *Here*: QA veterans rate top-10 question lists; low kappa means the rubric — not the raters — needs work.

A/B testing. Randomized controlled comparison of variants; needs power analysis (sample size for detectable effect) and guardrail metrics. *Here*: ranker

variants; but analyst traffic is low, so tests run long — hence heavy reliance on off-policy evaluation first.

Interleaving. Merge two rankers' outputs into one list and attribute clicks — more sensitive than A/B at low traffic. *Here:* useful for ranker tweaks when A/B would take months to reach significance.

Off-policy evaluation (IPS / SNIPS). Estimate a new policy's performance from logs of the old policy, correcting for the logging policy's action probabilities. SNIPS (self-normalized IPS) reduces variance. *Here:* every ranker iteration is scored on exploration logs before shadow deployment — cheap, no user impact.

Guardrail metrics. Metrics that must not regress even if the primary improves: abstention rate, citation precision, latency, validation reject rate. *Here:* a ranker with higher hit rate but collapsed diversity or soaring abstention doesn't ship.

L. Serving & infrastructure

API Gateway + Lambda + Fargate. Gateway: managed ingress (auth, throttling, WAF). Lambda: event-driven, scales to zero — perfect for thin auth/routing. Fargate: serverless containers for the stateful orchestrator (connection pools, model clients). *Here:* Gateway → Lambda (auth, context hash) → Fargate orchestrator. Right tool per layer.

WebSocket / SSE streaming. Full-duplex (WebSocket) or server-push (SSE) for incremental results. *Here:* answers stream as ready — ranked questions return in <500ms, answers follow. Never block ranking on generation.

Caching (Redis/ElastiCache). Key by context hash; TTL 1h; invalidated on material-change events. *Here:* the reason p99 < 2s is achievable — most inspection-mode requests are cache or precompute hits.

Precompute / materialization. Compute expensive results ahead of time (nightly + event-triggered). *Here:* per-site top-50 questions refreshed nightly and

on new deltas. Trades storage and staleness risk for latency — the right trade when QPS is low and contexts change slowly.

Timeouts, retries, exponential backoff + jitter. Bound every downstream call; retry with growing delays; jitter prevents thundering herds. *Here:* OpenAI calls: 30s timeout, 3 retries, backoff with jitter; generation failures degrade to retrieval+template candidates — never a total failure.

Circuit breaker. Stop calling a failing dependency for a cooldown period; fail fast to fallback. *Here:* if the embedding API degrades, serve from cache/precompute and alert — the analyst still gets yesterday's predictions.

Autoscaling & load shedding. Scale on queue depth/latency (not just CPU); shed low-priority work first under overload. *Here:* parsers scale on SQS depth; under extreme load, precompute jobs yield to live traffic.

Percentiles (p50/p99/p999). Averages hide tail suffering; SLOs are set on p99/p999. *Here:* p99 < 2s for re-rank; p999 allowed higher because generation is async. Always ask "p99 of what, measured where."

M. Streaming & MLOps

Event streaming (Kinesis / MSK). Durable, ordered-per-key, replayable event logs. Kinesis Firehose batches to S3 with zero ops. *Here:* interaction events (predicted_qids, asked_qids, thumbs, latency_ms, model_version) stream through Firehose → Parquet lake. Schema-registry enforced — event format changes are versioned, never silent.

SageMaker Pipelines. Orchestrated ML workflows: data prep → train → evaluate → register, with lineage tracking. *Here:* weekly ranker retraining + on-drift triggers; every run records data version, code version, metrics — reproducibility by construction.

Model Registry. Versioned model store with approval stages (pending → staging → production). *Here:* rankers, classifiers; promotion requires passing offline gates + off-policy eval; rollback is one API call.

Deployment strategies: shadow / canary / blue-green. Shadow: new model scores live traffic, results logged but unused (zero risk). Canary: 5% of traffic, watch guardrails. Blue-green: two full environments, instant switchover/rollback. *Here:* ranker goes shadow → 5% canary → full; index rebuilds are blue-green.

Drift detection. Feature drift (PSI, KS tests on distributions), label/concept drift (hit-rate decay), embedding drift. *Here:* nightly jobs compare recent vs reference windows; alarms page before users notice. Scheduled backtests on recent months catch slow decay.

Monitoring dashboards. Golden signals per component: traffic, errors, latency, saturation — plus ML-specific: hit rate, abstention, citation precision, feature freshness. *Here:* CloudWatch dashboards; one pane per system; alerts route to the owning team with runbook links.

N. Security & compliance

Encryption (KMS). Envelope encryption: data keys encrypt data, master keys in KMS encrypt data keys, with rotation and audit. *Here:* S3, DynamoDB, Neptune, OpenSearch — encrypted at rest; TLS in transit. Key policies separate duties (ML engineers can't decrypt the audit log).

IAM least privilege. Every component gets exactly the permissions it needs, nothing more; roles, not long-lived keys. *Here:* the parser Lambda can write to the chunk bucket but not read the audit log; the orchestrator can read the index but not the raw PII lake.

VPC & endpoints. Private subnets, no public IPs on data-plane components; VPC endpoints for AWS services; tightly allow-listed egress for OpenAI API calls. *Here:*

LLM calls never traverse the open internet unlogged — every call goes through the NAT with domain allow-listing and request logging.

Secrets Manager. Rotating, audited secret storage — never env vars or code.

Here: the OpenAI API key, with automatic rotation and least-privilege retrieval.

Audit logging (CloudTrail + immutable app log). CloudTrail records every AWS API call; the application log records every prediction/answer `{input_hash, model_versions, evidence_ids, timestamp, user}` to WORM S3. *Here:* the Part-11-style traceability story — reproduce any decision months later.

RBAC & ACLs at retrieval. Role-based access on internal documents, enforced as pre-filters in the retrieval query — not as post-generation filtering (which leaks via model behavior). *Here:* chunk-level `access_tier` tags; the ranker and generator never see unauthorized content.

PII redaction. Detect and mask personal data before it enters ML pipelines (Comprehend PII + LLM verification pass). *Here:* 483 narratives and training records are redacted before embedding; raw PII lives only in the source systems.

Zero data retention (ZDR). Contractual guarantee the API provider doesn't retain prompts. *Here:* required for any internal GxP content sent to OpenAI; public regulatory text is exempt. Always state this in interviews — it shows you think about data boundaries.

O. Reliability

Health checks & graceful degradation. Liveness vs readiness probes; when a dependency fails, serve reduced functionality. *Here:* ranker down → serve cached/precomputed lists with a staleness banner; embeddings down → BM25-only retrieval. The analyst always gets *something* useful.

RPO / RTO & DR. Recovery Point Objective (max data loss) and Recovery Time Objective (max downtime). *Here:* pilot-light second region; RPO 1h (event lake is

the source of truth, replayable), RTO 4h for the analyst UI. State these numbers in the interview — interviewers notice when you don't.

Chaos & runbooks. Game-days for failure injection; runbooks for every alarm (symptom → diagnosis → mitigation → escalation). *Here:* "OpenSearch indexing lag" runbook: check queue depth → scale OCUs → if still lagging, pause backfill, protect live traffic.

P. Cost engineering

Token economics. Know your per-1k-token input/output prices; blended cost per task. *Here:* extraction and judging are input-heavy (cheap); generation is output-heavy. Batch API halves offline costs; caching kills repeat spend. Rough steady state: low single-digit \$k/month, LLM-dominated.

Batch API (50% discount). Async, 24h turnaround. *Here:* backfills, synthetic data, bulk judging, embedding backfills — everything not on the live path.

Spot / serverless. Spot instances for fault-tolerant batch (60–90% off); serverless (Lambda, OpenSearch Serverless, Fargate) to pay per use at low/variable traffic. *Here:* Textract-batch on Batch+Spot; serving on Fargate+Lambda; no idle GPU bills.

Right-sizing & OCUs. OpenSearch Serverless bills per OCU-hour; Neptune per instance-hour. *Here:* separate collections (hot question bank vs warm archive), autoscaling bounds, and lifecycle policies so you're not paying hot prices for cold data.

PART 3 — One-page cheat sheet (memorize this)

- **One-liner:** Regulatory change monitoring → knowledge substrate → learning-to-rank question prediction → cited answers + evidence packs → feedback flywheel.
- **Three ML problems:** recall-optimized change detection; precision-optimized obligation extraction; learning-to-rank question prediction.
- **Data moat:** 483 reverse-engineering for labels; KG for applicability; interaction flywheel.
- **Anti-hallucination stack:** structured outputs → span grounding → LLM critic → NLI gate → abstention.
- **Ranker:** 11 features, LambdaMART, IPS debiasing, isotonic calibration, MMR, exploration slots.
- **Eval:** offline nDCG@10 on temporal splits; online hit rate@10; off-policy SNIPS before shipping.
- **Serving:** precompute + cache + stream; p99 < 2s questions; answers async.
- **Trade-offs to voice:** recall > precision (detection); graph + vectors (not either/or); LambdaMART > neural ranker (iteration speed); abstention is a feature.
- **Hardest parts:** label scarcity, KG maintenance, calibrated trust (predictions assistive, never oracular).
- **Build order:** alerts (6 wks) → RAG copilot → KG + retrieval prediction → generative ranker + inspection mode → hardening.

Good luck. Walk in, clarify for 5 minutes, draw the four boxes, and let them pull you into the deep dives — that's where this document lives in your head.